

Cloud 函式：自動將 CSV 資料匯入 Google 試算表

來源：<https://codelabs.developers.google.com/codelabs/cloud-function2sheet?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將瞭解如何透過 Cloud 函式來回應上傳至 Cloud Storage 的 CSV 檔案，並填入 Google 試算表

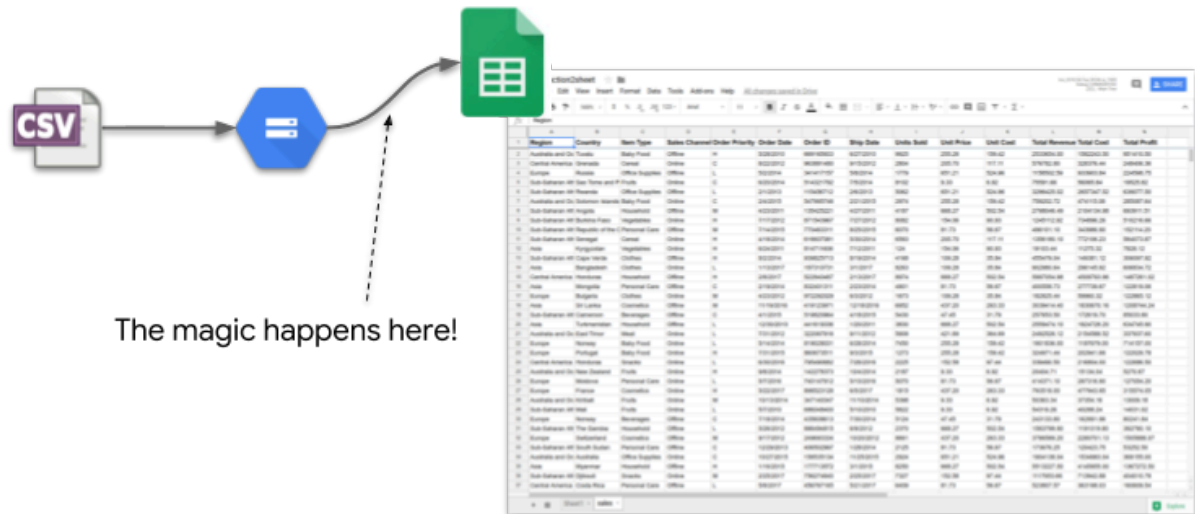
目錄

1. 簡介
2. 設定和需求
3. 建立及設定 Google 試算表，並啟用 API
4. 建立儲存空間值區
5. 建立 Cloud 函式
6. 設定驗證和 Google Sheets API
7. 使用 Sheets API 建立空白試算表
8. 從儲存空間 CSV 檔案讀取資料
9. 填入新建立的工作表
10. 整合所有內容並測試流程
11. 大功告成！拆除基礎架構
12. 後續步驟

步驟 1 · 約 2 分鐘

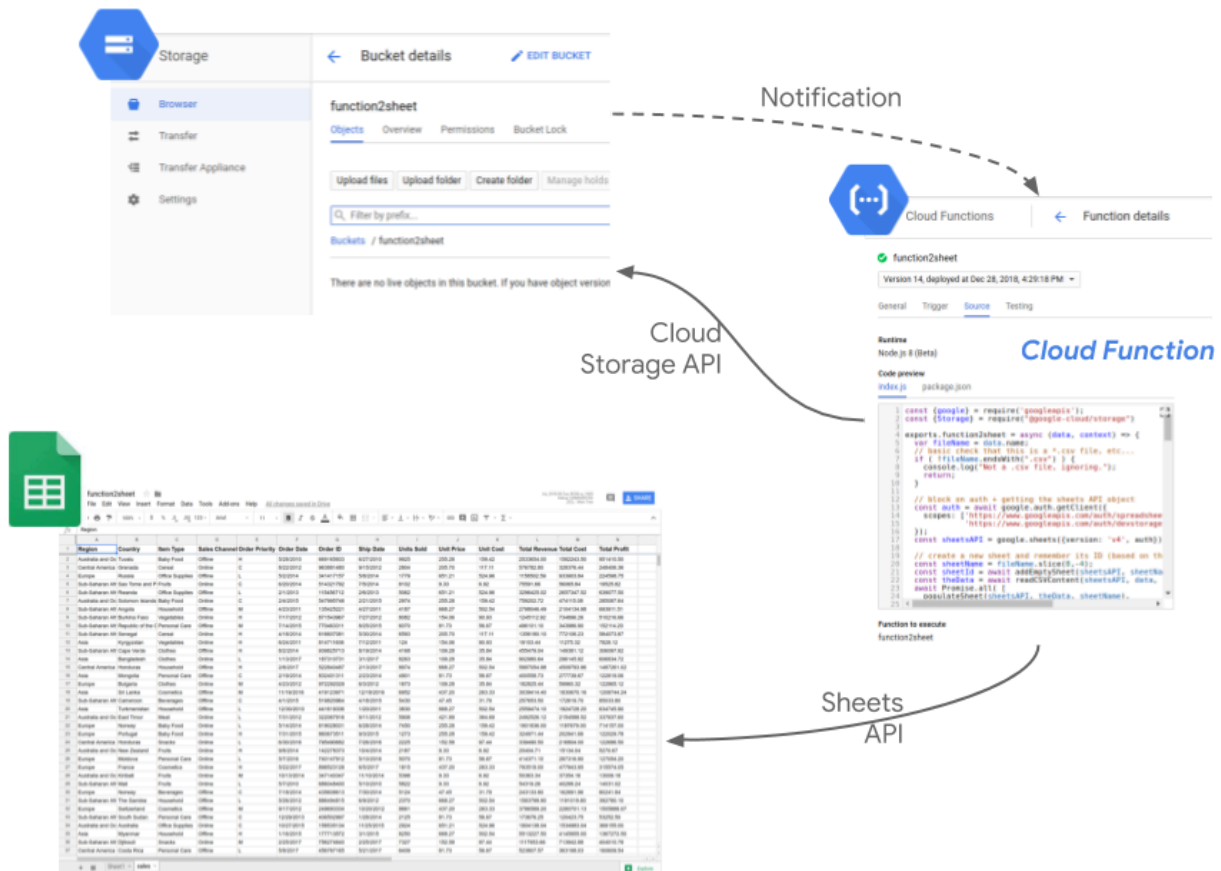
1. 簡介

本程式碼研究室的目標是協助您瞭解如何編寫 [Cloud Function](#)，以便對上傳至 [Cloud Storage](#) 的 CSV 檔案做出反應、讀取檔案內容，並使用 [Sheets API](#) 更新 Google 試算表。



這可視為自動執行原本需要手動操作的「匯入為 CSV」步驟。這樣一來，您就能在資料 (可能由其他團隊產生) 可用時，立即在試算表中分析資料。

實作方式如下：



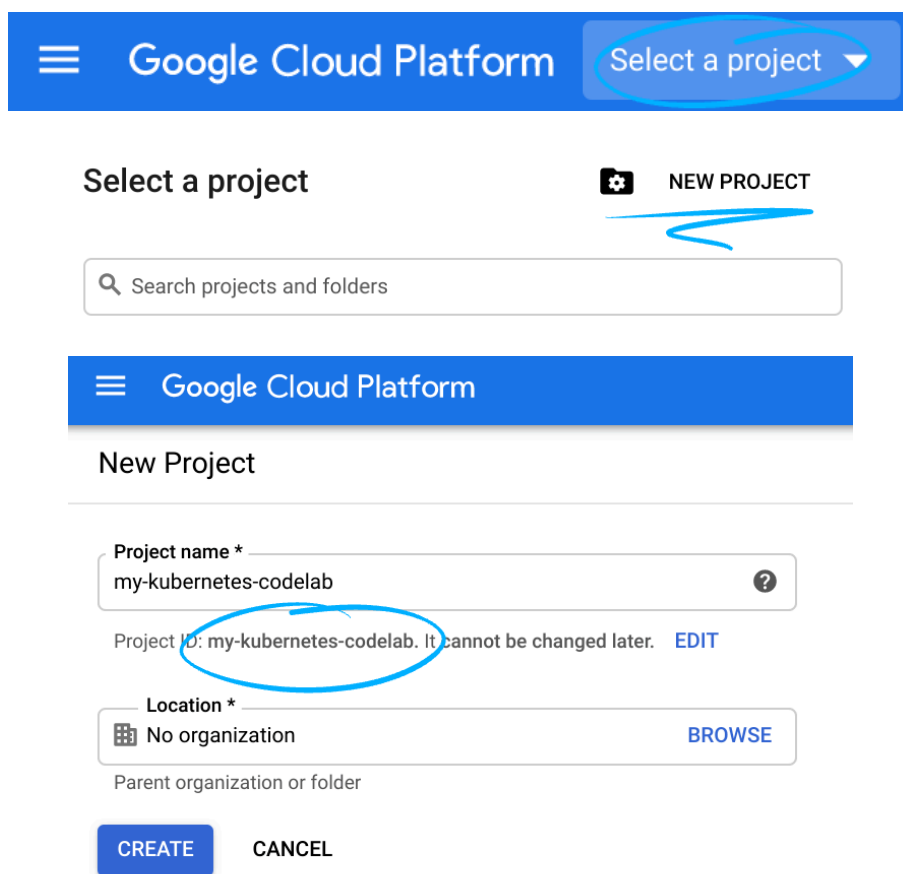
步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。



The screenshot displays the Google Cloud Platform interface for creating a new project. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu. Below this, the 'Select a project' section features a search bar labeled 'Search projects and folders' and a 'NEW PROJECT' button. The 'New Project' form is shown below, with fields for 'Project name *' (containing 'my-kubernetes-codelab'), 'Project ID' (containing 'my-kubernetes-codelab', which is circled in blue and has a note 'It cannot be changed later.'), and 'Location *' (set to 'No organization'). There are also 'CREATE' and 'CANCEL' buttons at the bottom.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

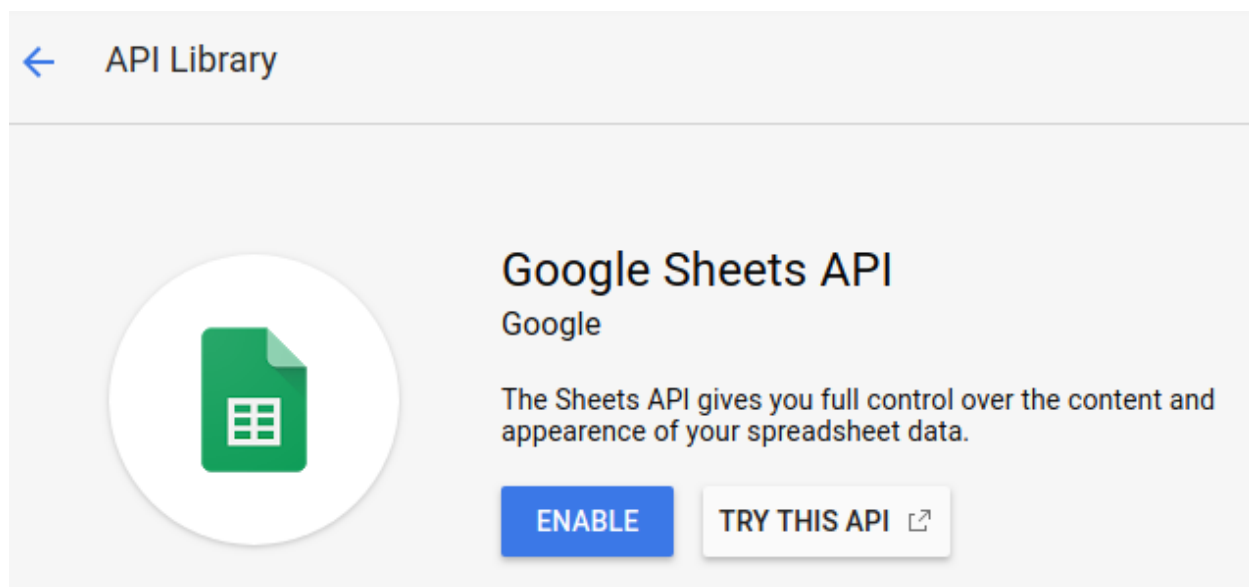
步驟 3 · 約 2 分鐘

3. 建立及設定 Google 試算表，並啟用 API

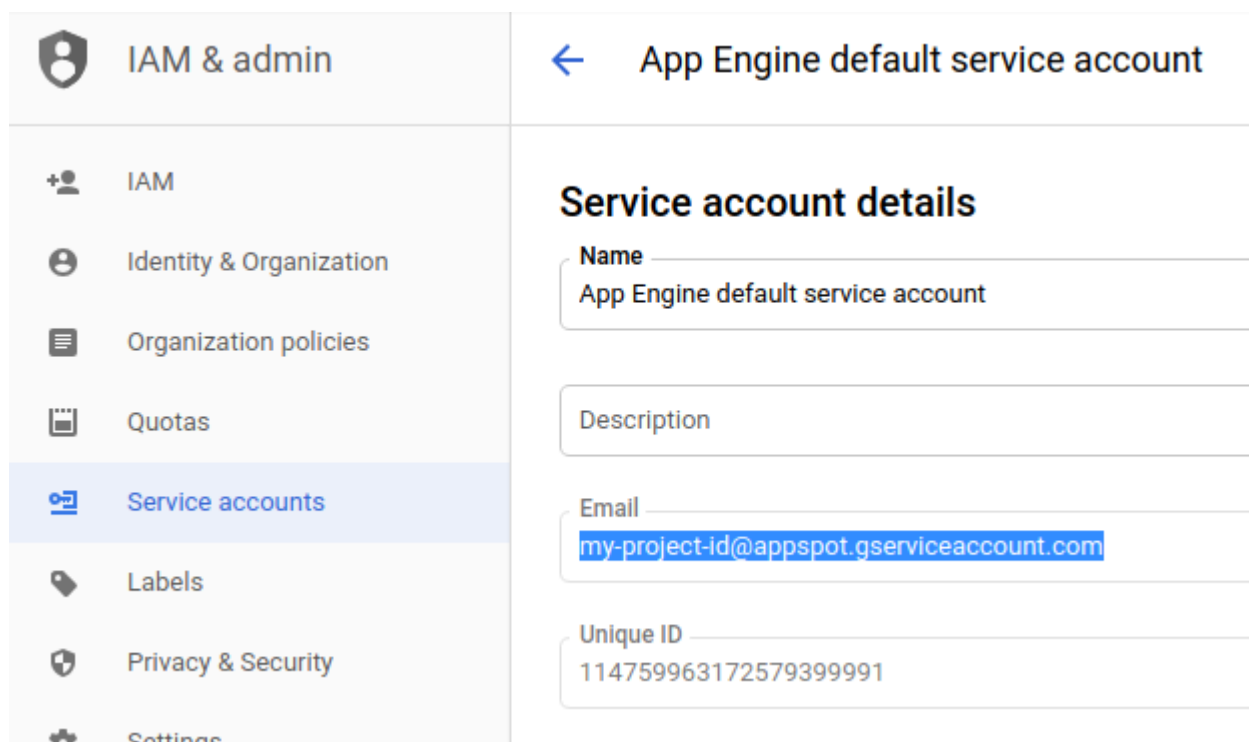
首先，請建立新的 Google 試算表文件 (這份試算表可屬於任何使用者)。建立後，請記下其 ID，我們稍後會將其做為所編寫函式的環境變數：

https://docs.google.com/spreadsheets/d/1ni3AC3wkDKjbGhaL38G3fspbQhqi5hakjh2Uwt_elj8o/edit#gid=0

前往 [GCP 控制台](#)，依序點選「API 和服務」和「API 程式庫」部分，在新建立的專案中啟用 Google Sheets API：



在「IAM 與管理」專區中，前往「服務帳戶」，並記下 App Engine 預設服務帳戶的電子郵件地址。格式應為 `your-project-id@appspot.gserviceaccount.com`。當然，您也可以建立專用於這項動作的服務帳戶。



最後，使用「共用」按鈕將試算表的編輯權授予這個服務帳戶：

Share with people and groups



 my-project-id@appspot-gserviceaccount.com 

Editor ▾

☐ Notify people

[Feedback?](#)

Cancel

Share

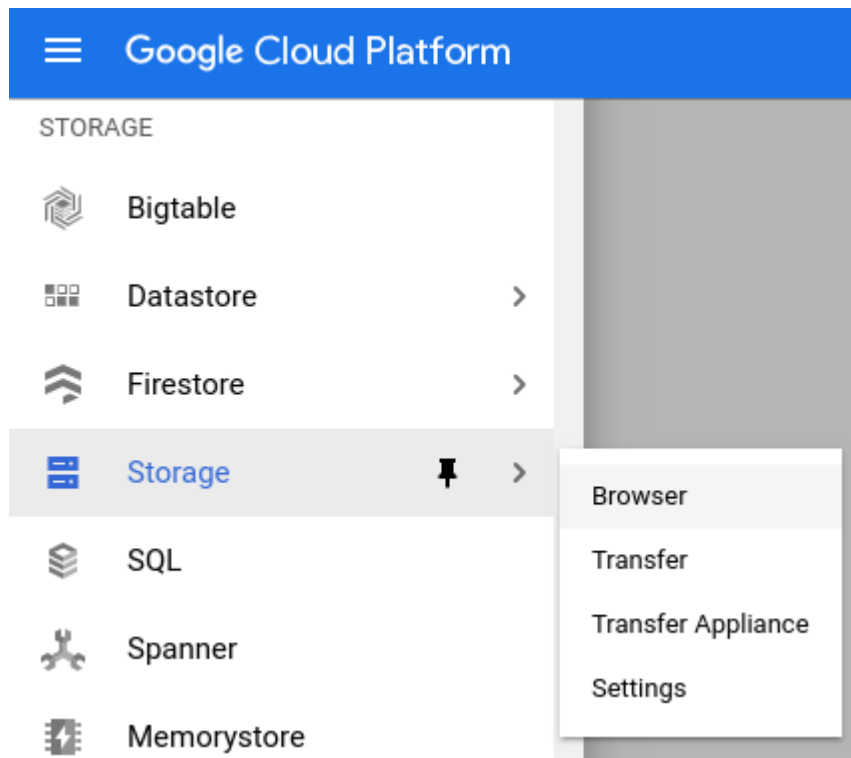
完成這項設定後，我們就能編寫 Cloud Function，並將其設定為使用這個服務帳戶。這個應用程式將可寫入我們剛才建立的試算表文件。

步驟 4 · 約 1 分鐘

4. 建立儲存空間值區

現在要建立 bucket，供 Cloud Function 監控新的 CSV 檔案。

在控制台中，使用左側選單前往「儲存空間」...



... 並建立名為 `csv2sheet-POSTFIX` 的新 bucket (將 POSTFIX 替換為不重複的內容)，其餘設定則全部保留預設值：

[←](#) Create a bucket

- Name your bucket**

Pick a **globally unique, permanent name**. [Naming guidelines](#)

csv2sheet-alexismj

Tip: Don't include any sensitive information

CONTINUE
- Choose where to store your data**
- Choose a default storage class for your data**
- Choose how to control access to objects**
- Advanced settings (optional)**

CREATE CANCEL

注意：您可以自行選擇位置、位置類型和儲存空間類別。這應該不會影響其餘程式碼研究室。如有疑問，請使用預設值。

步驟 5 · 約 3 分鐘

5. 建立 Cloud 函式

現在可以建立名為 `csv2sheet` 的 Cloud Function，在檔案上傳至特定 Cloud Storage bucket 時觸發。程式碼將以 Node.js 8 編寫，並使用 Cloud 控制台的內嵌編輯器，以 `async` 函式執行：

Cloud Functions

Create function

Name *

csv2sheet

?

Memory allocated *

256 MiB

▼

Trigger

Cloud Storage

▼

Event Type *

Finalize/Create

▼

?

Bucket *

csv2sheet

BROWSE

Source code

☒ Inline editor

☐ ZIP upload

☐ ZIP from Cloud Storage

☐ Cloud Source repository

Runtime

Node.js 8

▼

請務必將觸發條件設為「Cloud Storage」，並將值區名稱調整為您在上一步建立的名稱。

此外，請將我們即將編寫的函式進入點更新為 `csv2sheet`：

Function to execute *

csv2sheet

?

現在請將函式主體變更為：

1. 使用 Cloud Storage 和 Google 試算表 API
2. 將 `csv2sheet` 函式標示為 `async`
3. 從 Cloud Storage 事件中繼資料取得 `fileName`，並為要建立的新工作表衍生名稱：

```
const {google} = require("googleapis");
const {Storage} = require("@google-cloud/storage")

exports.csv2sheet = async (data, context) => {
  var fileName = data.name;
  // basic check that this is a *.csv file, etc...
  if (!fileName.endsWith(".csv")) {
    console.log("Not a .csv file, ignoring.");
    return;
  }
  // define name of new sheet
  const sheetName = fileName.slice(0, -4);

  // TODO!
};
```

如我們稍後會看到的，這裡必須使用 `async` 才能使用 `await`。

建立這項函式時，有幾個重要選項 (按一下畫面底部的「更多」連結)：

- 使用下拉式選單選取上述 **服務帳戶**
- 定義名為 `SPREADSHEET_ID` 的 **環境變數**，該變數應與您先前建立的試算表文件相符：

Advanced options

Region ?

us-central1

Timeout ?

60

seconds

Service account ?

App Engine default service account

☐ Retry on failure ?

Environment variables ?

Name

Value

SPREADSHEET_ID

1nI3AC3wkDKjbGhaL38G3fspbQ8y



+ Add variable

最後一個設定步驟如下，這是 `package.json` 內容，其中 Cloud Storage 和 Google 試算表 API 是我們要使用的兩個依附元件 (請使用控制台的內嵌編輯器 PACKAGE.JSON 分頁)：

```
{
  "name": "csv2sheet",
  "version": "0.0.42",
  "dependencies": {
    "googleapis": "^51.0.0",
    "@google-cloud/storage": "^5.0.1"
  }
}
```

按照上述說明完成所有設定後，請點按「建立」！函式應會在短時間內建立及部署完成。

注意：本實驗室討論的所有後續程式碼，都必須新增至我們剛才建立的單一 Cloud 函式。

步驟 6 · 約 2 分鐘

6. 設定驗證和 Google Sheets API

使用內嵌編輯器在 Cloud Function 中撰寫任何其他程式碼之前，我們需要使用適當的 Storage 和 Sheet 範圍建立 Google Client API (請注意，這是 `async` 函式的一部分)。

在控制台的函式編輯器中，按一下「EDIT」，然後將下列程式碼新增至 `csv2sheet` 函式的主體：

```
// block on auth + getting the sheets API object
const auth = await google.auth.getClient({
  scopes: [
    "https://www.googleapis.com/auth/spreadsheets",
    "https://www.googleapis.com/auth/devstorage.read_only"
  ]
});
```

接著，我們可以建立 Sheets API 用戶端：

```
const sheetsAPI = google.sheets({version: 'v4', auth});
```

7. 使用 Sheets API 建立空白試算表

有了 Sheets API 用戶端，我們可以在文件中建立簡單的新試算表，但在此之前，請先快速瞭解詞彙：

- **試算表**是實際文件，並由其 ID 參照 (如上所述，且顯示在文件網址中)
- **工作表**是文件中的其中一個分頁，可透過名稱 (分頁名稱) 或工作表建立時產生的 ID 參照

瞭解這點後，以下函式會使用 Sheets API 用戶端，在位置 2 (通常是預設的「Sheet1」之後) 建立空白工作表，其中包含 26 欄、2000 列，並凍結第一列 (使用內嵌編輯器將其新增至函式)：

```
function addEmptySheet(sheetsAPI, sheetName) {
  return new Promise((resolve, reject) => {
    const emptySheetParams = {
      spreadsheetId: process.env.SPREADSHEET_ID,
      resource: {
        requests: [
          {
            addSheet: {
              properties: {
                title: sheetName,
                index: 1,
                gridProperties: {
                  rowCount: 2000,
                  columnCount: 26,
                  frozenRowCount: 1
                }
              }
            }
          }
        ]
      }
    };
    sheetsAPI.spreadsheets.batchUpdate( emptySheetParams, function(err, response) {
      if (err) {
        reject("The Sheets API returned an error: " + err);
      } else {
        const sheetId = response.data.replies[0].addSheet.properties.sheetId;
        console.log("Created empty sheet: " + sheetId);
        resolve(sheetId);
      }
    });
  });
}
```

請注意，我們並非將試算表的參照硬式編碼，而是依賴先前建立的 `SPREADSHEET_ID` 環境變數。

我們需要記住 `sheetId`，以便對這個特定工作表提出進一步要求。此外，工作表名稱不得重複，如果已有「`sheetName`」這個名稱的工作表，系統就會建立失敗。

Sheets API 中的 `batchUpdate` 函式是與文件互動的常見方式，詳情請參閱[這篇文章](#)。

8. 從儲存空間 CSV 檔案讀取資料

現在我們有地方可以傾印資料，接下來請在內嵌編輯器中進一步開發 Cloud Function，並使用 Cloud Storage API 從剛上傳的檔案中擷取實際資料，然後儲存在字串中：

```
function readCSVContent(sheetsAPI, file, sheetName) {
  return new Promise((resolve, reject) => {
    const storage = new Storage();
    let fileContents = new Buffer('');
    storage.bucket(file.bucket).file(file.name).createReadStream()
      .on('error', function(err) {
        reject('The Storage API returned an error: ' + err);
      })
      .on('data', function(chunk) {
        fileContents = Buffer.concat([fileContents, chunk]);
      })
      .on('end', function() {
        let content = fileContents.toString('utf8');
        console.log("CSV content read as string : " + content );
        resolve(content);
      });
  });
}
```

步驟 9 · 約 4 分鐘

9. 填入新建立的工作表

現在，請使用相同的 Google 試算表用戶端 API 和剛才收集的資料，填入我們建立的試算表。我們也會趁這個機會為試算表的欄新增一些樣式 (變更頂端列的字型大小並設為粗體)：


```

function populateAndStyle(sheetsAPI, theData, sheetId) {
  return new Promise((resolve, reject) => {
    // Using 'batchUpdate' allows for multiple 'requests' to be sent in a single batch.
    // Populate the sheet referenced by its ID with the data received (a CSV string)
    // Style: set first row font size to 11 and to Bold. Exercise left for the reader: resize
columns
    const dataAndStyle = {
      spreadsheetId: process.env.SPREADSHEET_ID,
      resource: {
        requests: [
          {
            pasteData: {
              coordinate: {
                sheetId: sheetId,
                rowIndex: 0,
                columnIndex: 0
              },
              data: theData,
              delimiter: ",",
            }
          },
          {
            repeatCell: {
              range: {
                sheetId: sheetId,
                startRowIndex: 0,
                endRowIndex: 1
              },
              cell: {
                userEnteredFormat: {
                  textFormat: {
                    fontSize: 11,
                    bold: true
                  }
                }
              },
              fields: "userEnteredFormat(textFormat)"
            }
          }
        ]
      }
    };

    sheetsAPI.spreadsheets.batchUpdate(dataAndStyle, function(err, response) {
      if (err) {
        reject("The Sheets API returned an error: " + err);
      } else {
        console.log(sheetId + " sheet populated with " + theData.length + " rows and column
style set.");
        resolve();
      }
    });
  });
}

```

```
});  
}
```

這段程式碼應新增至 Cloud Function，現在已完成 99%！

請注意，資料和樣式會合併為多個 `requests`，並呼叫單一 Google 試算表 API `batchUpdate`。這樣一來，更新作業會更有效率，且具有不可分割性。

另請注意，我們定義的編輯範圍與建立的工作表大小相符。也就是說，如果內容超過 26 欄 (建立試算表時使用的 `columnCount` 值)，就會因這個特定程式碼而失敗。

如果一切順利，此時您可以：

1. 儲存更新後的函式
 2. 將 CSV 檔案拖曳至值區
 3. 試算表就會顯示對應資料！
-

步驟 10 · 約 4 分鐘

10. 整合所有內容並測試流程

我們剛討論的函式呼叫可以在原始 `csv2sheet` 函式中，做為連續的封鎖呼叫：

```
const sheetId = await addEmptySheet(sheetsAPI, sheetName);
const theData = await readCSVContent(sheetsAPI, data, sheetName);
await populateAndStyle(sheetsAPI, theData, sheetId);
```

如需完整的函式原始碼，請參閱[這篇文章](#) (一次取得所有函式可能比較容易)。

一切就緒後，只要將 CSV 檔案上傳至正確的 bucket，試算表就會更新，並新增含有檔案內容的工作表。如果您沒有現成的 CSV 檔案，可以下載[這個範例](#)。

function2sheet

Region	Country	Item Type	Sales Channel	Order Priority	Order Date	Order ID	Ship Date	Units Sold	Unit Price	Unit Cost	Total Revenue	Total Cost	Total Profit
Australia and Oc Tuvalu	Baby Food	Offline	H		5/28/2010	669165933	6/27/2010	9925	255.28	159.42	2533654.00	1582243.90	951410.50
Central America Grenada	Cereal	Online	C		8/22/2012	963881480	9/15/2012	2804	205.70	117.11	576782.80	328376.44	248406.36
Europe Russia	Office Supplies	Offline	L		5/2/2014	341417157	5/6/2014	1779	651.21	524.96	1158502.59	933903.84	224598.75
Sub-Saharan Afr Sao Tome and P	Fruits	Online	C		6/20/2014	514321792	7/5/2014	8102	9.33	6.92	75591.66	56065.84	19525.82
Sub-Saharan Afr Rwanda	Office Supplies	Offline	L		2/1/2013	115456712	2/6/2013	5062	651.21	524.96	3296425.02	2657347.52	639077.50
Australia and Oc Solomon Islands	Baby Food	Online	C		2/4/2015	547995746	2/21/2015	2974	255.28	159.42	759202.72	474115.08	285087.64
Sub-Saharan Afr Angola	Household	Offline	M		4/23/2011	135425221	4/27/2011	4187	668.27	502.54	2798046.49	2104134.98	693911.51
Sub-Saharan Afr Burkina Faso	Vegetables	Online	H		7/17/2012	871543967	7/27/2012	8082	154.06	90.93	1245112.92	734896.26	510216.66
Sub-Saharan Afr Republic of the C	Personal Care	Offline	M		7/14/2015	770463311	8/25/2015	6070	81.73	56.67	496101.10	343986.90	152114.20
Sub-Saharan Afr Senegal	Cereal	Online	H		4/19/2014	616607081	5/30/2014	6593	205.70	117.11	1356180.10	772106.23	584073.87
Asia Kyrgyzstan	Vegetables	Online	H		6/24/2011	814711606	7/12/2011	124	154.06	90.93	19103.44	11275.32	7828.12
Sub-Saharan Afr Cape Verde	Clothes	Offline	H		8/2/2014	939825713	8/19/2014	4168	109.28	35.84	455479.04	149381.12	306097.92
Asia Bangladesh	Clothes	Online	L		1/13/2017	187310731	3/1/2017	8263	109.28	35.84	902980.64	296145.92	606834.72
Central America Honduras	Household	Offline	H		2/8/2017	522840487	2/13/2017	8974	668.27	502.54	5997054.98	4509793.96	1487261.02
Asia Mongolia	Personal Care	Offline	C		2/19/2014	832401311	2/23/2014	4901	81.73	56.67	400558.73	277739.67	122819.06
Europe Bulgaria	Clothes	Online	M		4/23/2012	972292029	6/3/2012	1673	109.28	35.84	182825.44	59960.32	122865.12
Asia Sri Lanka	Cosmetics	Offline	M		11/19/2016	419123971	12/18/2016	6952	437.20	263.33	3039414.40	1830670.16	1208744.24
Sub-Saharan Afr Cameroon	Beverages	Offline	C		4/1/2015	519820964	4/18/2015	5430	47.45	31.79	257653.50	172619.70	85033.80
Asia Turkmenistan	Household	Online	L		12/30/2010	441619336	1/20/2011	3830	668.27	502.54	2559474.10	1924728.20	634745.90
Australia and Oc East Timor	Meat	Online	L		7/31/2012	322067916	9/11/2012	5908	421.89	364.69	2492526.12	2154588.52	337937.60
Europe Norway	Baby Food	Online	L		5/14/2014	819028031	6/28/2014	7450	255.28	159.42	1901836.00	1187679.00	714157.00
Europe Portugal	Baby Food	Online	H		7/31/2015	860873511	9/3/2015	1273	255.28	159.42	324971.44	202941.66	122029.78
Central America Honduras	Snacks	Online	L		6/30/2016	795490882	7/26/2016	2225	152.58	97.44	339490.50	216804.00	122686.50
Australia and Oc New Zealand	Fruits	Online	H		9/8/2014	142278373	10/4/2014	2187	9.33	6.92	20404.71	15134.04	5270.67
Europe Moldova	Personal Care	Online	L		5/7/2016	740147912	5/10/2016	5070	81.73	56.67	414371.10	287316.90	127054.20
Europe France	Cosmetics	Online	H		5/22/2017	898523128	6/5/2017	1815	437.20	263.33	793518.00	477943.95	315574.05
Australia and Oc Kiribati	Fruits	Online	M		10/13/2014	347140347	11/10/2014	5398	9.33	6.92	50363.34	37354.16	13009.18
Sub-Saharan Afr Mali	Fruits	Online	L		5/7/2010	686048400	5/10/2010	5822	9.33	6.92	54319.26	40288.24	14031.02
Europe Norway	Beverages	Offline	C		7/18/2014	435608613	7/30/2014	5124	47.45	31.79	243133.80	162891.96	80241.84
Sub-Saharan Afr The Gambia	Household	Offline	L		5/26/2012	886494815	6/9/2012	2370	668.27	502.54	1583799.90	1191019.80	392780.10
Europe Switzerland	Cosmetics	Offline	M		9/17/2012	249693334	10/20/2012	8661	437.20	263.33	3786589.20	2280701.13	1505888.07
Sub-Saharan Afr South Sudan	Personal Care	Offline	C		12/29/2013	406502997	1/28/2014	2125	81.73	56.67	173676.25	120423.75	53252.50
Australia and Oc Australia	Office Supplies	Online	C		10/27/2015	158535134	11/25/2015	2924	651.21	524.96	1904138.04	1534983.04	369155.00
Asia Myanmar	Household	Offline	H		1/16/2015	177713572	3/1/2015	8250	668.27	502.54	5513227.50	4145955.00	1367272.50
Sub-Saharan Afr Djibouti	Snacks	Online	M		2/25/2017	756274640	2/25/2017	7327	152.58	97.44	1117953.66	713942.88	404010.78
Central America Costa Rica	Personal Care	Offline	L		5/8/2017	456767165	5/21/2017	6409	81.73	56.67	523807.57	363198.03	160609.54

試著將多個檔案上傳至 bucket，看看會發生什麼事！

步驟 11 · 約 1 分鐘

11. 大功告成！拆除基礎架構

開玩笑的，我們不需要拆除基礎架構，因為這一切都是以無伺服器方式完成！

您可以視需要刪除建立的 Cloud Function 和儲存空間，甚至是整個專案。

步驟 12 · 約 0 分鐘

12. 後續步驟

本程式碼研究室到此結束。您已瞭解如何透過 Cloud Function 監聽 Cloud Storage bucket 的上傳作業，並使用適當的 API 更新 Google 試算表。

以下是幾個後續步驟：

- 參閱 Cloud Functions [使用指南](#) (內含部分最佳做法)
- 完成其中一個 Cloud Functions [教學課程](#)
- 進一步瞭解 [Google Sheets API](#)

如果您在本程式碼研究室遇到任何問題，歡迎使用左下角的連結回報。

歡迎提供意見！
